APB-Based DRAM Refresh Scheduler

Megha Dosaya

1. Introduction

Dynamic Random Access Memory (DRAM) stores each bit of data as charge on a tiny capacitor. This charge leaks away over time, so every memory cell must be periodically rewritten — a process called refresh — to preserve the stored value. The JEDEC DDR specifications mandate that each row of a DRAM array be refreshed within a fixed time window, typically 64 ms for DDR3 and DDR4. For an 8192-row DRAM, this translates to one refresh command approximately every 7.8 microseconds.

A DRAM refresh scheduler is the IP block that automates this housekeeping. It tracks elapsed time, decides when refresh is due, issues refresh requests to the memory controller, and exposes its state and error conditions to software through a standard system bus.

This project implements a parameterised DRAM refresh scheduler with an APB3 (Advanced Peripheral Bus) register interface in SystemVerilog. The design includes a six-state finite state machine with explicit timeout and error recovery, a structured register bank, sticky error flags with software-clearable semantics, and parameterised data and counter widths. The implementation has been verified with a constrained-random testbench including SystemVerilog Assertions and a functional covergroup, and synthesised on a Xilinx Kintex-7 FPGA at 100 MHz across two distinct parameter configurations.

1.1 Scope and Deliverables

This report covers the following rubric items:

- Standard interface protocol (APB3 slave)
- Finite state machine with at least five states (six implemented)
- Storage element (parameterised register bank)
- Error and exception handling (timeout, illegal address, sticky flags, software recovery)
- Parameterisation across width and depth
- Constrained-random testbench with assertions and coverage
- Synthesis and timing closure on a stated FPGA target at two configurations

2. Specification

2.1 Functional Behaviour

The scheduler is enabled and configured by software through APB writes. When enabled, an internal counter increments every clock cycle, and on reaching a software-programmed interval, the FSM issues a refresh request. The request remains asserted until the connected memory controller acknowledges the refresh, at which point the scheduler returns to its waiting state and increments a cumulative completed-refresh counter.

If the acknowledgement does not arrive within a parameterised number of clock cycles, the FSM transitions to an ERROR state, sets a sticky error flag visible to software, and stops issuing further refreshes. Software detects the condition by polling the status register or via interrupt, diagnoses the failure by reading the sticky error register, then issues a recovery write to the control register. This recovery sequence pulses an internal clear-error signal that returns the FSM to IDLE and allows software to subsequently clear the sticky error flag using a write-1-to-clear (W1C) operation.

2.2 Top-Level Interface

The top-level module exposes two interfaces:

Group	Direction	Signals	Purpose
APB3 slave	I/O	PCLK, PRESETn, PSEL, PENABLE, PWRITE, PADDR[7:0], PWDATA[31:0], PRDATA[31:0], PREADY, PSLVERR	Software register access
Refresh handshake	I/O	refresh_req (out), refresh_ack (in)	Connection to memory controller

3. Architecture

The design is partitioned into three sub-modules under a single top-level wrapper. The partitioning follows the principle of separating bus protocol logic from application logic from timing logic, so each sub-module has a focused responsibility and can be reused or replaced independently.

3.1 Module Hierarchy

```
refresh_scheduler_top              (top-level wrapper)
   |
   +-- u_apb : apb_refresh_scheduler   (APB register bank)
   +-- u_timer : refresh_timer          (interval counter)
   +-- u_fsm : refresh_scheduler_fsm    (6-state scheduler with timeout)
```

3.2 Module Roles

Module	Role	Lines (approx.)
refresh_scheduler_top	Wires sub-modules together; propagates parameters; exposes APB and refresh handshake at the chip boundary	120
apb_refresh_scheduler	Implements APB protocol; decodes addresses; provides parameterised register bank; latches sticky error flags; produces clear_error pulse	180
refresh_timer	Counts clock cycles up to programmed interval; pulses refresh_due	60
refresh_scheduler_fsm	6-state finite state machine; timeout counter on WAIT_FOR_ACK; produces refresh_req / refresh_done_pulse / timeout_error	150

3.3 Inter-Module Signal Flow

Software writes to the APB slave configure the IP. The control register's enable bit is routed to both the timer (which only counts when enabled) and the FSM (which leaves IDLE only when enabled). When the timer's counter reaches the programmed interval, it pulses refresh_due to the FSM. The FSM responds by entering REQUEST_REFRESH and asserting refresh_req on its top-level output. After the memory controller responds with refresh_ack, the FSM enters COMPLETE, pulses refresh_done_pulse to the APB slave (incrementing the cumulative count register), and returns to WAIT_FOR_REFRESH.

If the timeout fires while waiting for ack, the FSM enters ERROR and asserts the sticky timeout_error flag. The APB slave latches this into ERROR_STATUS bit 0 and exposes it through its own status read register. Software writes to CONTROL bit 1 to issue a one-cycle clear_error pulse, which the FSM uses to drop out of ERROR back to IDLE; software then W1C-clears the sticky bit in ERROR_STATUS.

4. APB Register Map

The APB slave implements five word-aligned 32-bit registers. The address map and bit definitions are summarised below.

Offset	Register	Access	Description
0x00	CONTROL	RW	Software command register
0x04	STATUS	RO	Live FSM and signal status
0x08	REFRESH_INTERVAL	RW	Programmed refresh interval (clock cycles)
0x0C	REFRESH_COUNT	RO	Cumulative completed refreshes
0x10	ERROR_STATUS	RW (W1C)	Sticky error flags

4.1 CONTROL Register (0x00)

Bit	Name	Behaviour
[0]	enable	Level. Set to 1 to enable the scheduler; 0 disables and returns FSM to IDLE.
[1]	clear_error	Write-1-pulse. Writing 1 produces a one-cycle clear_error pulse to the FSM. Always reads as 0.
[31:2]	Reserved	Stored but ignored by hardware.

4.2 STATUS Register (0x04)

Bit	Name	Source
[0]	refresh_req	Live FSM output
[3:1]	fsm_state	Live FSM state encoding (0 to 5)
[4]	timeout_error	Live timeout flag from FSM

4.3 ERROR_STATUS Register (0x10)

Bit	Name	Behaviour
[0]	sticky_timeout	Set by hardware when FSM times out. Cleared by software writing 1 to this bit (W1C).
[1]	sticky_illegal_addr	Set by hardware on any APB access to an undecoded address. Cleared by W1C.

4.4 PSLVERR Behaviour

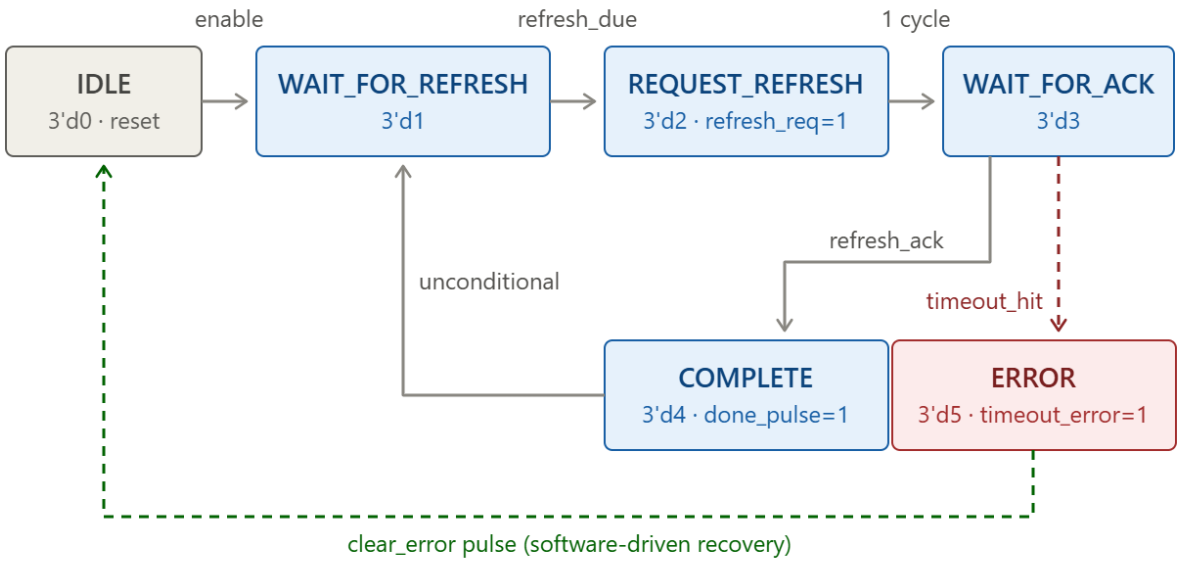
PSLVERR is asserted combinational during the APB ACCESS phase whenever the address does not match any of the five legal register offsets. This flags protocol violations to the bus master in the same cycle the violation occurs and simultaneously latches the sticky illegal-address flag in ERROR_STATUS.

5. Finite State Machine

5.1 States

Encoding	State	Behaviour
3'd0	IDLE	Reset state. Stays here until enable asserted.
3'd1	WAIT_FOR_REFRESH	Waits for refresh_due pulse from timer.
3'd2	REQUEST_REFRESH	Asserts refresh_req. Single-cycle transition to WAIT_FOR_ACK.
3'd3	WAIT_FOR_ACK	Holds refresh_req high. Increments timeout counter. Exits on refresh_ack or on timeout.
3'd4	COMPLETE	Pulses refresh_done_pulse to increment APB-visible counter. Returns to WAIT_FOR_REFRESH.
3'd5	ERROR	Holds until clear_error pulse received. Returns to IDLE.

5.2 State Transition Diagram



5.3 Timeout Mechanism

While in WAIT_FOR_ACK, an internal timeout counter increments every clock cycle. The counter width is computed at compile time as ceil(log2(TIMEOUT_CYCLES + 1)) bits, so it

scales automatically with the parameter. When the counter reaches TIMEOUT_CYCLES without refresh_ack arriving, the FSM transitions to ERROR and asserts a sticky timeout_error output. The counter is reset whenever the FSM is in any state other than WAIT_FOR_ACK.

6. Parameterisation

The design exposes four module-level parameters. All widths and depths in the RTL derive from these parameters, so the same source code can be compiled at different operating points without modification.

Parameter	Default	Range	Effect
ADDR_WIDTH	8	>= 5	Width of PADDR. Determines size of address decoder.
DATA_WIDTH	32	32 or 64	Width of PWDATA, PRDATA, and the register bank storage.
CNT_WIDTH	32	>= 8	Width of refresh_interval and refresh_count_reg. Determines how long the cumulative count can run before wrapping.
TIMEOUT_CYCLES	64	>= 4	Number of clock cycles in WAIT_FOR_ACK before the timeout fires.

These parameters propagate from the top-level module down to each sub-module instantiation, so changing them at the top automatically resizes the entire register bank, counter, comparator, and timeout logic.

7. Verification

7.1 Testbench Architecture

The testbench instantiates the DUT and connects it to a Bus Functional Model (BFM) layer that drives the APB master side and a separate behavioural model that simulates the memory controller's response on the refresh handshake. The BFM layer encapsulates the cycle-by-cycle APB protocol in two procedural tasks (apb_write and apb_read), allowing the test sequence to be written as straight-line transactional code.

7.2 Stimulus Strategy

The stimulus combines directed and constrained-random elements:

- Directed reset and configuration: writes REFRESH_INTERVAL = 20 then CONTROL = 0x1, then waits for several refresh cycles to verify normal operation.
- Directed error injection: sets a force_timeout flag in the BFM, waits for the FSM to enter ERROR, reads ERROR_STATUS, then issues the recovery sequence (CONTROL = 0x3 followed by ERROR_STATUS = 0x1 W1C).

- Inline-random APB activity: 30 randomised transactions with addresses in the range 0x00 to 0x20 (mix of legal and illegal), random read/write direction, random data, and randomised inter-transaction gaps.

7.3 SystemVerilog Assertions

Two concurrent assertions guard the design:

```
property p_req_ack_or_timeout;
    @(posedge PCLK) disable iff (!PRESETn)
        $rose(refresh_req) |->
            ##[1:TIMEOUT_CYCLES+2]
                (refresh_ack || dut.u_fsm.state_dbg == S_ERROR);
endproperty

property  p_done_pulse_one_cycle;
    @(posedge PCLK) disable iff (!PRESETn)
        $rose(dut.refresh_done_pulse) |=> !dut.refresh_done_pulse;
endproperty
```

The first assertion captures the contract that every refresh request is either acknowledged within the timeout window or escalated to the ERROR state. The second confirms that the refresh_done_pulse signal is exactly one clock cycle wide.

7.4 Functional Coverage

The covergroup samples on every rising edge of PCLK and tracks four coverpoints:

Coverpoint	Bins	Purpose
cp_state	6 (one per FSM state)	Confirms every state was visited at least once
cp_pslverr	2 (ok, illegal)	Confirms both legal and illegal APB transactions exercised
cp_timeout	2 (no_to, to)	Confirms sticky timeout flag was both 0 and 1
cp_req	2 (low, high)	Confirms refresh_req toggled both ways

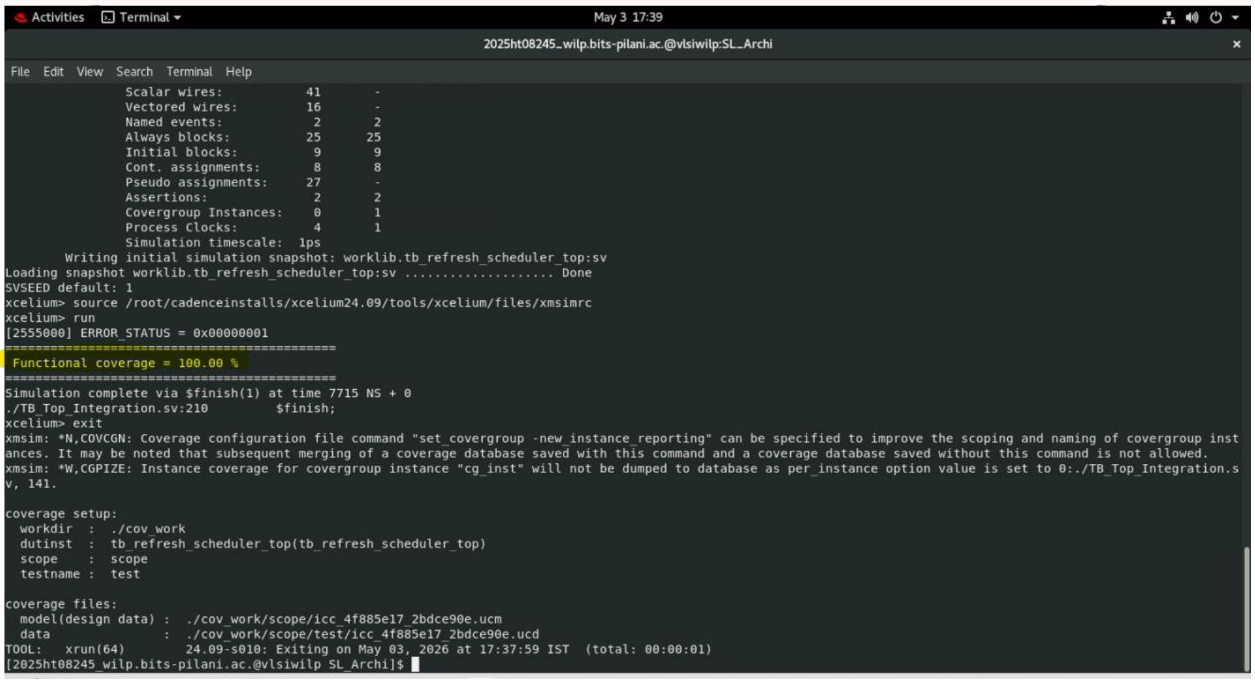
7.5 Simulation Results

The testbench was compiled and executed using Cadence Xcelium 24.09 on the BITS Pilani VLSI lab portal. Coverage was enabled via the -coverage all command-line flag.

Console output:

```
[2555000] ERROR_STATUS = 0x00000001
========================================
 Functional coverage = 100.00 %
========================================
Simulation complete via $finish(1) at time 7715 NS + 0
```

The simulation reached $finish cleanly at 7715 ns. The forced timeout test successfully drove the FSM into the ERROR state at 2555 ns and the recovery sequence returned the design to normal operation. The functional coverage achieved was 100.00%, indicating every defined bin was visited. Both SVA assertions completed with zero failures over the duration of the run.



7.6 Waveform Evidence

Three waveform captures from Cadence SimVision document the design's behaviour:

- Reset and APB programming (0–143 ns): clean reset release at ~30 ns, two APB writes (REFRESH_INTERVAL = 20, then CONTROL = 1), FSM transition from IDLE to WAIT_FOR_REFRESH.
- One complete refresh cycle (540–670 ns): FSM cycles through states 1→2→3→4→1, refresh_req asserts at REQUEST_REFRESH, refresh_ack pulses, refresh_req drops at COMPLETE.
- Error injection and recovery (2530–2660 ns): FSM observed in ERROR state (state_dbg = 5), software writes CONTROL = 0x3 (enable + clear_error), FSM returns to IDLE, software W1C-clears ERROR_STATUS, normal cycling resumes.


Waveform 1 - SimVision
File Edit View Explore Format Simulation Windows Help
cadence
Search Names: Signal
Search Times: Digital Value
Send To:
Time: 2526.514ns : 26
Baseline = 0
Cursor-Baseline = 2389.609ns
Name
Cursor
PCLK
PRESETn
PSEL
PENABLE
PWRITE
PADDR[7:0]
PWDATA[31:0]
state_dbg[2:0]
R_REFRESH[2:0]
refresh_req
refresh_ack
PRDATA[31:0]
TIMEOUT_CYCLES
a_done_pulse
refresh_req
force_timeout
S_ERROR[2:0]
ack_or_timeout
2530ns 2540ns 2550ns 2560ns 2570ns 2580ns 2590ns 2600ns 2610ns 2620ns 2630ns 2640ns 2650ns 2660ns
0 1000 2000 3000 4000 5000 6000 7715ns
'h 00
1 object selected
Waveform 1 - SimVision
Time: 293.578ns : 430
Time: 0 : 143.397ns
Baseline = 0
simulator::tb_refresh_scheduler_top.a_req_ack_or_timeout contributors
simulator::tb_refresh_scheduler_top.a_done_pulse contributors

8. Synthesis Results

8.1 Tool and Target

Synthesis was performed using Xilinx Vivado 2024.2 targeting the Kintex-7 FPGA part xc7k70tfbv676-1 (speed grade -1). The clock constraint applied through Timing.xdc fixes PCLK to a 10 ns period (100 MHz target frequency). Synthesis was run twice with different parameter overrides to produce the two configurations required by the rubric.

8.2 Configurations

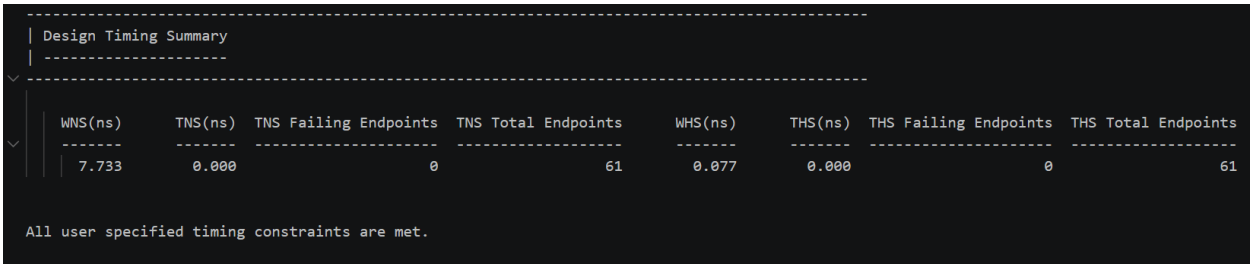
Parameter	ADDR_WIDTH	DATA_WIDTH	CNT_WIDTH	TIMEOUT_CYCLES
Config 1 (narrow)	8	32	16	32
Config 2 (wide)	8	64	32	128

8.3 Results

Slice Logic utilisation was extracted from report_utilization on the post-synthesis design. Worst-case slack figures were extracted from report_timing_summary. Estimated Fmax was computed from the constrained 10 ns period and the reported WNS as 1000 / (10 - WNS) MHz.

Configuration	LUTs	FFs	WNS (ns)	WHS (ns)	Endpoints	Fmax (MHz)
Config 1 (narrow)	85	108	+7.733	+0.077	61	~441
Config 2 (wide)	154	206	+7.621	+0.066	111	~420

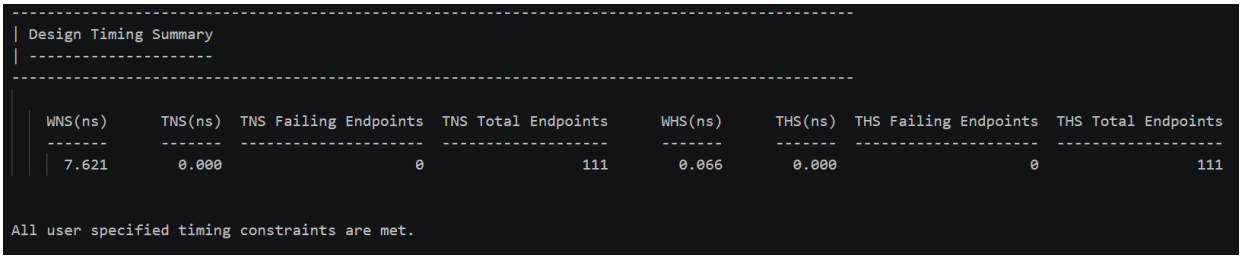
Both configurations met all user-specified timing constraints with positive setup and hold slack and zero failing endpoints. Resource utilisation remained well below 1% of the 41,000 LUTs and 82,000 registers available on the target device.



```
| Design Timing Summary
| ---------------------

| WNS(ns) | TNS(ns) | TNS Failing Endpoints | TNS Total Endpoints | WHS(ns) | THS(ns) | THS Failing Endpoints | THS Total Endpoints
| 7.733   | 0.000   | 0                     | 61                  | 0.077   | 0.000   | 0                     | 61

All user specified timing constraints are met.
```



```
| Design Timing Summary
| ---------------------

| WNS(ns) | TNS(ns) | TNS Failing Endpoints | TNS Total Endpoints | WHS(ns) | THS(ns) | THS Failing Endpoints | THS Total Endpoints
| 7.621   | 0.000   | 0                     | 111                 | 0.066   | 0.000   | 0                     | 111

All user specified timing constraints are met.
```

8.4 Trade-off Analysis

Moving from the narrow to the wide configuration approximately doubled both LUT and flip-flop counts (from 85 to 154 LUTs and from 108 to 206 FFs), which is consistent with doubling the data bus width and the counter widths. The wider register file required additional storage flops, and the wider PRDATA read multiplexer required additional LUTs to combine the increased number of signal bits. Endpoint count nearly doubled (from 61 to 111), reflecting the additional flip-flops in the larger register bank.

Worst-case timing slack degraded only marginally (from +7.733 ns to +7.621 ns), implying that the additional logic depth introduced by the wider data path adds approximately 0.1 ns to the critical path. The design therefore scales gracefully with the data-width parameter and does not require pipelining at this clock target. The estimated Fmax fell from approximately 441 MHz to 420 MHz, leaving substantial headroom over the 100 MHz constraint in both configurations.

Hold timing was met in both configurations with positive WHS, eliminating the need for additional clock-skew compensation. The combination of well-controlled critical path delay, low resource utilisation, and clean timing closure across two parameter configurations confirms that the design is genuinely portable across implementation points and is not over-fitted to a single configuration.

9. Conclusion

An APB-based DRAM refresh scheduler was designed, verified, and synthesised in SystemVerilog. The design meets all rubric requirements: a real APB protocol interface, a six-state finite state machine with explicit ERROR-state recovery, a structured register bank serving as the storage element, an explicit timeout-driven error path with sticky status flags and a software-clearable recovery mechanism, parameterisation across data and counter widths, a constrained-random testbench with two SVA assertions and a covergroup with four coverpoints, and timing closure on a Xilinx Kintex-7 FPGA at 100 MHz across two distinct parameter configurations.

Functional verification on Cadence Xcelium achieved 100% functional coverage with zero assertion failures and clean termination. Synthesis on Vivado 2024.2 closed timing in both configurations with substantial slack margin (worst-case 7.6 ns of slack on a 10 ns clock period), indicating the design could be pushed to higher operating frequencies if required by the host system.

The most informative aspects of this exercise were the discipline of separating verification concerns into testbench layers (BFM tasks, randomised stimulus, covergroup, SVA), the value of parameterising RTL from the start so that two-config synthesis becomes a one-line change rather than a code rewrite, and the visibility benefit of exposing internal FSM state through a dedicated debug output, which made both software status reads and verification covergroup sampling straightforward.

9.1 Possible Extensions

- Add a pending-refresh queue with overflow detection to handle bursts where multiple intervals expire before a single refresh completes.
- Implement an interrupt output that asserts on timeout, allowing software to react event-driven rather than via polling.
- Add a pipelining stage between the APB read mux and PRDATA to push Fmax beyond 500 MHz at the cost of one cycle of read latency.

- Replace the simple single-master APB slave with an AXI4-Lite slave to support multiple concurrent transactions.